



MAJOR PROJECT
SOFTWARE REQUIREMENTS SPECIFICATION

TITLE: Explainable AI for Interpreting Black-Box Machine Learning Models

MediXAI — Explainable AI for Drug

Submitted By:

Name	Sap Id:	Roll No:	Branch
Manas Arora	500109627	R2142221204	CSE AI/ML(B-5 Non-Hons)

Approved by

Under the guidance of:

Mr. Debanjan Ghosh
(Project Guide)

Cluster Head

School Of Computer Science
UNIVERSITY OF PETROLEUM & ENERGY STUDIES,
DEHRADUN- 248007. Uttarakhand

Table of Contents

TOPIC	PAGE NO.
1. Introduction	
• 1.1 Purpose of this Document	3
• 1.2 Scope of the Product	3
• 1.3 Literature Review	4-5
• 1.4 Definitions, Acronyms, and Abbreviations	5
• 1.5 References	5-6
• 1.6 Overview	6
2. Overall Description	
• 2.1 Product Perspective	7
• 2.2 Product Functions	7-8
• 2.3 User Characteristics/Personas	8
• 2.4 Operating Environment	9
• 2.5 Design and Implementation Constraints	9
• 2.6 Assumptions and Dependencies	9-10
3. Specific Requirements	
• 3.1 External Interface Requirements (User, Hardware, Software, Communication)	10-11
• 3.2 Functional Requirements (Input/Output processing, Features)	12-13
• 3.3 Non-Functional Requirements (Performance, Safety, Security, Quality Attributes)	13-14
• 3.4 Database Requirements	14
4. System Modeling/Architecture	
• 4.1 Use Case Diagrams/Scenarios	15-16
• 4.2 Data Flow Diagrams/Class Diagrams	16-20
5. Appendices	
• 5.1 Appendix A: Glossary	21
• 5.2 Appendix B: Analysis Model	22-23
• 5.3 Appendix C: Issues List	23-24

School Of Computer Science
UNIVERSITY OF PETROLEUM & ENERGY STUDIES,
DEHRADUN- 248007. Uttarakhand

1. Introduction

1.1 Purpose of this Document

The purpose of this Software Requirements Specification (SRS) document is to provide a comprehensive and structured description of the MediXAI system. MediXAI is an Explainable Artificial Intelligence-based web application designed to analyze drug reviews, provide sentiment insights, and recommend treatments based on user-reported symptoms.

This lack of interpretability raises several important concerns. In sensitive domains such as healthcare diagnosis, financial risk assessment, or educational analytics, stakeholders need clear explanations for AI decisions. Without transparency, users may hesitate to trust AI predictions, and organizations may face regulatory and ethical challenges.

This document serves as a reference for developers, evaluators, and stakeholders by clearly defining the system's functionality, requirements, constraints, and expected behavior. It ensures that all aspects of the system are well understood before deployment and evaluation. Additionally, it acts as a foundation for future enhancements and scalability of the system.

1.2 Scope of the Product

MediXAI is a full-stack AI-powered platform that integrates Natural Language Processing (NLP), Machine Learning (ML), and Explainable AI (XAI) techniques to assist users in understanding drug effectiveness and potential risks.

The system provides the following core functionalities:

- **Drug Review Analysis:** Classifies user reviews into Positive, Neutral, or Negative sentiments using a BERT-based model.
- **Explainable AI Insights:** Uses SHAP-based explanations (in local environment) to highlight influential words contributing to predictions.
- **Drug Insights Dashboard:** Displays benefits, limitations, sentiment distribution, and keyword-based insights for selected drugs and conditions.
- **Drug Comparison Feature:** Allows users to compare two drugs based on real-world user sentiment and feedback.
- **Symptom-Based Prediction (CheckUp):** Predicts possible diseases based on user-input symptoms using a Random Forest model and suggests medications and precautions.

The system is intended for educational, analytical, and research purposes. It is not designed to replace professional medical diagnosis or treatment.

1.3 Literature Review

Explainable Artificial Intelligence (XAI) has emerged as an important research area due to the increasing adoption of machine learning models in critical decision-making domains. Traditional machine learning models, particularly deep learning and ensemble methods, often operate as black boxes, making it difficult to interpret how predictions are generated. This lack of transparency has raised concerns about trust, accountability, fairness, and ethical use of artificial intelligence systems.

Tjoa and Guan provided one of the most comprehensive surveys on Explainable Artificial Intelligence, categorizing explanation methods into intrinsic interpretability and post-hoc interpretability techniques ^[1]. Their work highlighted how explainability improves transparency and user trust in machine learning systems, particularly in high-stakes applications. Similarly, Linardatos et al. presented a detailed review of interpretability methods, focusing on feature importance analysis, visualization techniques, and model-agnostic explanation approaches ^[2]. Their study emphasized the importance of explainable AI in sensitive domains such as healthcare and finance where decision transparency is critical.

Ribeiro et al. introduced LIME (Local Interpretable Model-agnostic Explanations), which explains individual predictions by approximating complex models locally using simpler interpretable models ^[3]. This approach allows users to understand specific predictions without

needing full access to the internal model structure. Another widely adopted method is SHAP (SHapley Additive exPlanations), proposed by Lundberg and Lee, which uses game theory principles to assign importance values to input features ^[4]. SHAP provides consistent and theoretically grounded explanations, making it popular for interpreting machine learning models.

Doshi-Velez and Kim emphasized the need for a rigorous scientific framework to evaluate interpretability in machine learning systems ^[5]. Their research highlighted the importance of human-centered explanations and standardized evaluation metrics. Adadi and Berrada conducted a comprehensive review discussing challenges in explainable AI, including the trade-off between interpretability and predictive accuracy ^[6]. They emphasized that achieving both high accuracy and strong interpretability remains a key research challenge.

Guidotti et al. proposed a taxonomy of explanation methods and explored different strategies for explaining black-box models ^[7]. Their work outlined future research directions, including improving explanation reliability and developing standardized evaluation frameworks. Rudin argued that in high-risk applications, inherently interpretable models should be preferred over black-box models with post-hoc explanations ^[8]. This perspective highlights the ongoing debate regarding whether explainability should be integrated into model design or applied after model development.

Samek et al. explored explainability in deep neural networks, discussing visualization techniques

that help interpret complex deep learning models ^[9]. Their work demonstrated how explainable AI can improve understanding of model behavior and detect potential biases. Molnar provided a comprehensive overview of interpretable machine learning techniques, discussing both model-specific and model-agnostic approaches along with practical implementation considerations ^[10].

Overall, existing literature demonstrates that explainable AI significantly enhances transparency, user trust, and accountability in machine learning systems. However, challenges remain in balancing interpretability with predictive performance, developing standardized evaluation methods, and ensuring explanations are understandable to non-expert users. This project builds upon existing research by implementing an explainable AI framework that integrates predictive modeling with interpretability techniques, aiming to improve transparency and usability of artificial intelligence systems.

1.4 Definitions, Acronyms, and Abbreviations

- **AI (Artificial Intelligence):** Simulation of human intelligence in machines.
- **ML (Machine Learning):** A subset of AI that enables systems to learn from data.
- **NLP (Natural Language Processing):** Technique used to analyze and understand human language.
- **XAI (Explainable Artificial Intelligence):** AI systems that provide transparency in decision-making.
- **SHAP (SHapley Additive exPlanations):** A method used to interpret machine learning model predictions.
- **API (Application Programming Interface):** Interface enabling communication between frontend and backend systems.
- **UI (User Interface):** The graphical interface through which users interact with the system.
- **SRS (Software Requirements Specification):** Document describing system requirements and behavior.

1.5 References

The development of MediXAI is based on the following references:

- [1] Tjoa, E. & Guan, C. (2020). *A Survey on Explainable Artificial Intelligence (XAI)*. IEEE Transactions on Neural Networks and Learning Systems. DOI: <https://arxiv.org/abs/1907.07374>
- [2] Linardatos, P., Papastefanopoulos, V., & Kotsiantis, S. (2020). *Explainable AI: A Review of Machine Learning Interpretability Methods*. Entropy Journal. Retrieved from <https://www.mdpi.com/1099-4300/23/1/18>

- [3] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). *Why Should I Trust You? Explaining the Predictions of Any Classifier*. Proceedings of ACM SIGKDD. DOI: <https://arxiv.org/abs/1602.04938>
- [4] Lundberg, S. & Lee, S. (2017). *A Unified Approach to Interpreting Model Predictions (SHAP)*. Advances in Neural Information Processing Systems. Retrieved from <https://arxiv.org/abs/1705.07874>
- [5] Doshi-Velez, F. & Kim, B. (2017). *Towards a Rigorous Science of Interpretable Machine Learning*. Retrieved from <https://arxiv.org/abs/1702.08608>
- [6] Adadi, A. & Berrada, M. (2018). *Peeking Inside the Black-Box: A Survey on Explainable AI*. IEEE Access. DOI: <https://ieeexplore.ieee.org/document/8466590>
- [7] Devlin, J. et al. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*
- [8] Lundberg, S. M., & Lee, S. I. (2017). *A Unified Approach to Interpreting Model Predictions (SHAP)*
- Kaggle Medical Dataset for disease prediction and drug-related data
 - FastAPI Documentation: <https://fastapi.tiangolo.com/>
 - React Documentation: <https://reactjs.org/>
 - Scikit-learn Documentation: <https://scikit-learn.org/>

1.6 Overview

This document is organized into multiple sections to provide a detailed understanding of the MediXAI system:

- Section 1 introduces the system and its purpose.
- Section 2 provides an overall description of the product, including system perspective and user characteristics.
- Section 3 defines specific functional and non-functional requirements.
- Section 4 presents system architecture and modeling diagrams.
- Section 5 includes additional supporting information and appendices.

This structured approach ensures clarity, traceability, and completeness of the system design and requirements.

2. Overall Description

2.1 Product Perspective

MediXAI is a standalone, web-based application designed as a full-stack system integrating frontend, backend, and machine learning components. It operates as an intelligent analytical platform that bridges user interaction with advanced AI models.

The system follows a client-server architecture:

- The **frontend (React.js)** provides an interactive and user-friendly interface for data input, visualization, and navigation.
- The **backend (FastAPI)** handles API requests, processes inputs, and communicates with machine learning models.
- The **machine learning layer** consists of:
 - A BERT-based model for sentiment analysis of drug reviews
 - A Random Forest model for disease prediction based on symptoms

The system is designed to function independently without requiring integration with external hospital or healthcare databases. However, it is structured in a modular way, allowing future integration with real-world medical systems or APIs.

2.2 Product Functions

MediXAI provides multiple interconnected functionalities that collectively form an intelligent healthcare analytics system:

1. Drug Review Sentiment Analysis

- Accepts user-provided drug reviews
- Classifies sentiment into Positive, Neutral, or Negative
- Provides confidence scores for predictions

2. Explainable AI (XAI) Analysis

- Highlights important words influencing predictions (via SHAP in local setup)
- Generates human-readable explanations of model decisions

3. Drug Insights Dashboard

- Displays:
 - Sentiment distribution
 - Top keywords
 - Benefits and limitations
 - Severity of side effects

- Allows users to analyze specific drugs and conditions

4. Drug Comparison System

- Compares two drugs based on:
 - Positive sentiment percentage
 - Negative sentiment percentage
 - Neutral sentiment percentage
- Helps users evaluate alternative treatments

5. Symptom-Based Disease Prediction (CheckUp)

- Accepts user symptoms (text input)
- Converts input into structured symptom representation
- Predicts likely disease using Random Forest model
- Provides:
 - Disease description
 - Recommended medications
 - Precautions
 - Derived benefits and side-effect insights

2.3 User Characteristics / Personas

The system is designed for a broad range of users with varying technical expertise:

1. General Users / Patients

- Limited technical knowledge
- Interested in understanding drug effectiveness and side effects
- Uses dashboard and CheckUp features for decision support

2. Medical Students / Researchers

- Moderate technical understanding
- Interested in analyzing patterns in drug reviews
- Uses explainability and insights for academic purposes

3. Developers / Evaluators

- High technical expertise
- Interested in system architecture and model performance
- Evaluates system behavior and implementation

The interface is designed to be intuitive and accessible to all user types.

2.4 Operating Environment

The system operates in the following environment:

Frontend

- Browser-based application
- Compatible with modern browsers (Chrome, Edge, Firefox)
- Developed using React.js

Backend

- Python-based FastAPI server
- Runs locally or on cloud platforms (e.g., Render)

System Requirements

- Minimum:
 - 4 GB RAM
 - Python 3.9+
 - Node.js (for frontend)
- Recommended:
 - 8 GB RAM (for ML model handling)

2.5 Design and Implementation Constraints

The system is subject to the following constraints:

- **Model Size Constraints:**
Large models such as BERT increase memory and computation requirements.
- **Deployment Constraints:**
Free cloud platforms may not support heavy dependencies like SHAP and SciPy.
- **Dataset Limitations:**
The system relies on publicly available datasets, which may not fully represent real-world diversity.
- **Real-Time Processing:**
Some operations (e.g., SHAP explanations) are computationally expensive and may affect performance.
- **Medical Validity:**
The system does not replace professional medical advice and must include disclaimers.

2.6 Assumptions and Dependencies

Assumptions

- Users provide meaningful and relevant input (reviews or symptoms)
- Datasets used for training are reasonably representative
- Users understand that the system is for informational purposes

Dependencies

- Python libraries: FastAPI, PyTorch, Transformers, Scikit-learn
- Frontend libraries: React.js, Axios, Chart.js
- Pre-trained models (BERT and Random Forest)
- Availability of dataset files in the backend directory

The system depends on proper configuration of these components for correct operation.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interface Requirements

The MediXAI system provides a modern, interactive, and responsive web-based user interface designed using React.js. The interface is structured into multiple modules:

- **Home Page:** Provides an overview of the system and navigation to core features.
- **Dashboard Page:** Allows users to select drugs and conditions to view insights such as sentiment distribution, benefits, and limitations.
- **CheckUp Page:** Accepts natural language symptom input and returns predicted disease, medications, and precautions.
- **Explain Page:** Displays sentiment prediction for a given review along with explanation.
- **Help Page:** Provides guidance on how to use the system and explains key concepts like SHAP and AI.

UI Features:

- Dropdown-based selection for drugs and conditions
- Input fields for free-text symptom entry
- Interactive charts (Pie charts for sentiment distribution)
- Dynamic cards for benefits and limitations
- Responsive design supporting different screen sizes
- Theme support (light/dark mode)

3.1.2 Hardware Interface Requirements

The system does not require specialized hardware. It runs on standard computing devices:

- Desktop or Laptop (Recommended)
- Minimum 4 GB RAM

- Standard keyboard and mouse input devices

No external hardware components are required.

3.1.3 Software Interface Requirements

The system interacts with the following software components:

- **Frontend:** React.js application running in a browser
- **Backend:** FastAPI server handling API requests
- **Operating System:**
Windows 10/11 (64-bit), Ubuntu 20.04+, or macOS 11+
- **Programming Language:**
Python (Version 3.8 or later)
- **Development Environment:**
Jupyter Notebook / Google Colab (for experimentation and model testing)
Visual Studio Code or PyCharm (for development and integration)
- **Machine Learning Libraries:**
Scikit-learn (for traditional ML models)
TensorFlow or PyTorch (for deep learning models)
- **Explainable AI Libraries:**
SHAP (feature attribution and interpretability)
LIME (local explanation of predictions)
- **Data Processing Libraries:**
Pandas and NumPy (data handling and preprocessing)
- **Visualization Libraries:**
Matplotlib, Seaborn, or Plotly (for explanation visualization and analytics)

3.1.4 Communication Interface Requirements

- Protocol: HTTP/HTTPS
- Data Format: JSON
- API Calls:
 - GET requests for fetching drug data, conditions, and analysis
 - POST requests for prediction tasks (review sentiment and symptom-based prediction)

The system ensures smooth communication between frontend and backend through well-defined API endpoints.

3.2 Functional Requirements

The system provides the following core functionalities:

3.2.1 Drug Review Sentiment Analysis

- The system shall accept user-input drug reviews.
- The system shall preprocess the text input.
- The system shall classify the review into one of three categories:
 - Positive
 - Neutral
 - Negative
- The system shall return a confidence score for the prediction.
- The system shall generate a basic explanation using keyword-based logic if SHAP is unavailable.

3.2.2 Explainable AI Analysis

- The system shall highlight influential words affecting prediction.
- The system shall provide a human-readable explanation.
- The system shall fallback to keyword-based explanation in case of model limitations.

3.2.3 Drug Insights Dashboard

- The system shall allow users to select a drug and condition.
- The system shall filter relevant reviews from dataset.
- The system shall compute:
 - Sentiment distribution
 - Top keywords
 - Benefits and limitations
 - Severity levels
- The system shall display results using charts and visual cards.

3.2.4 Drug Comparison Feature

- The system shall accept two drug inputs and one condition.
- The system shall compute sentiment distribution for both drugs.
- The system shall display comparative results (percentage-based).
- The system shall handle cases where data is unavailable.

3.2.5 Symptom-Based Disease Prediction (CheckUp)

- The system shall accept symptom input in natural language.
- The system shall extract relevant symptoms from input text.
- The system shall convert symptoms into structured feature vector.
- The system shall predict disease using Random Forest model.
- The system shall return:
 - Predicted disease
 - Description
 - Recommended medications
 - Precautions
- The system shall optionally provide insights from review dataset.

3.2.6 Navigation and Interaction

- The system shall allow seamless navigation between pages.
- The system shall provide feedback during loading states.
- The system shall display error messages when required.

3.3 Non-Functional Requirements

3.3.1 Performance Requirements

- The system shall respond within 2–5 seconds for most requests.
- The system shall handle moderate user load efficiently.
- Model inference time should remain acceptable for user experience.

3.3.2 Usability Requirements

- The interface shall be intuitive and easy to use.
- Users shall not require technical knowledge to operate the system.
- The system shall provide clear instructions and feedback.

3.3.3 Reliability Requirements

- The system shall handle invalid inputs gracefully.
- The system shall provide fallback responses in case of model errors.
- The system shall maintain consistent output across similar inputs.

3.3.4 Security Requirements

- The system shall not store sensitive user data.
- Input validation shall be performed to prevent misuse.
- APIs shall handle malformed requests safely.

3.3.5 Maintainability Requirements

- The system shall follow modular architecture.
- Code shall be organized into frontend and backend components.
- The system shall allow easy updates and feature additions.

3.3.6 Scalability Requirements

- The system shall support future integration with larger datasets.
- The architecture shall allow deployment on scalable cloud platforms.

3.4 Database Requirements

The system uses a dataset-driven approach rather than a traditional database.

Data Sources:

- Drug review datasets (train.csv, test.csv)
- Medical datasets (symptoms, medications, precautions, descriptions)

Data Characteristics:

- Structured tabular data (CSV format)
- Preprocessed for model training and inference

Data Usage:

- Used for:
 - Sentiment analysis
 - Insight extraction
 - Disease prediction

Storage:

- Data is stored locally in backend directory
- No external database (e.g., SQL/NoSQL) is used

Future Scope:

- Integration with database systems for scalability and real-time data updates

4. System Modeling / Architecture

4.1 Use Case Diagrams / Scenarios

4.1.1 Overview

The MediXAI system is designed to interact with users through a series of well-defined use cases. These use cases represent how users engage with the system to perform specific tasks such as analyzing drug reviews, comparing drugs, and predicting diseases based on symptoms. The primary actor in the system is the **User**, who interacts with different modules of the application.

4.1.2 Primary Use Cases

1. Analyze Drug Reviews

- User selects a drug and condition from dropdown menus.
- System retrieves relevant reviews from dataset.
- System processes data and generates:
 - Sentiment distribution
 - Benefits and limitations
 - Keyword insights
- System displays results through charts and cards.

2. Explain Review Prediction

- User clicks on a specific review.
- System sends review text to backend.
- Backend model predicts sentiment.
- System generates explanation (SHAP or fallback logic).
- Results are displayed to the user.

3. Compare Drugs

- User inputs two drug names and a condition.
- System retrieves data for both drugs.
- System computes sentiment percentages.
- System displays comparison results.

4. Perform AI CheckUp (Symptom-Based Prediction)

- User enters symptoms in natural language.
- System extracts relevant symptom keywords.
- System converts input into structured feature vector.
- Random Forest model predicts disease.

- System retrieves:
 - Description
 - Medications
 - Precautions
- Results are displayed in structured format.

4.1.3 Use Case Diagram (Conceptual Description)

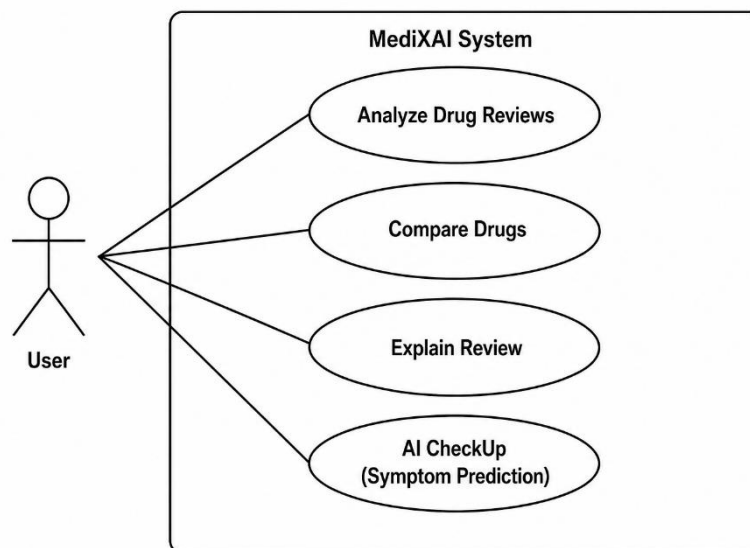
The use case diagram consists of:

- **Actor:** User
- **System:** MediXAI

Connections:

- User → Analyze Drug Reviews
- User → Compare Drugs
- User → Explain Review
- User → Perform CheckUp

Each use case interacts with backend APIs and machine learning models.



Img 1: Use Case Diagram

4.2 Data Flow Diagrams / Class Diagrams

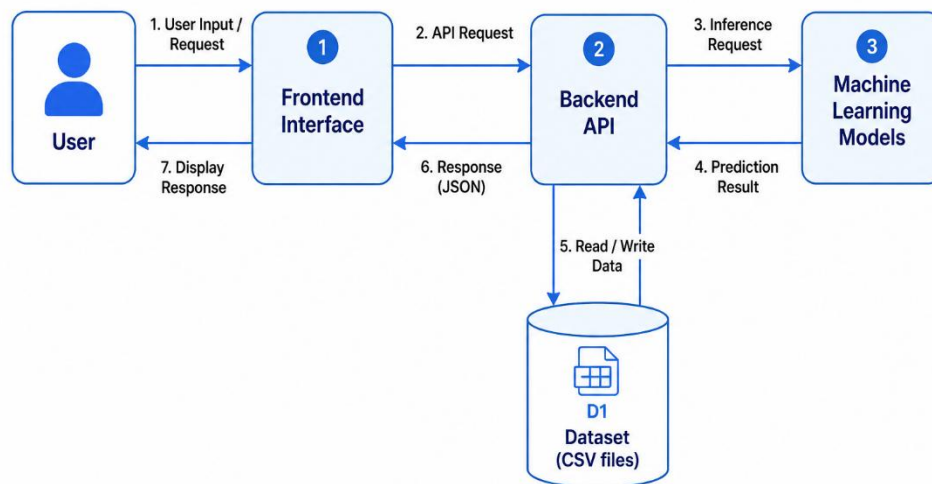
4.2.1 Data Flow Diagram (DFD Level 0)

At a high level, the system follows this data flow:

1. User provides input (review text, drug selection, or symptoms)
2. Input is sent to backend via API

3. Backend processes input using:
 - BERT model (for sentiment analysis)
 - Random Forest model (for disease prediction)
4. Backend retrieves relevant dataset information
5. Processed results are returned as JSON response
6. Frontend displays results using UI components

Level 0 Data Flow Diagram (DFD)



4.2.2 Data Flow Diagram (DFD Level 1)

Detailed flow:

For Sentiment Analysis:

- Input: Review text
- Process:
 - Tokenization
 - Model inference (BERT)
 - Probability calculation
 - Explanation generation
- Output:
 - Sentiment label
 - Confidence score
 - Explanation

For Dashboard Analysis:

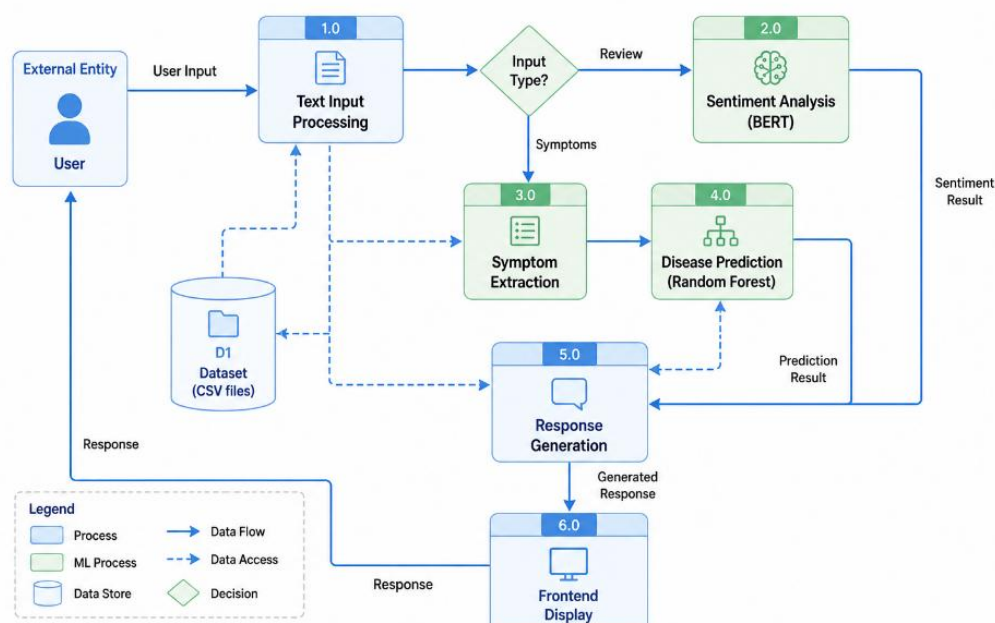
- Input: Drug + Condition

- Process:
 - Filter dataset
 - Compute sentiment counts
 - Extract keywords
 - Categorize benefits and limitations
- Output:
 - Charts
 - Insight cards

For CheckUp System:

- Input: Symptoms (text)
- Process:
 - Symptom extraction
 - Feature vector creation
 - Random Forest prediction
 - Lookup medical datasets
- Output:
 - Disease prediction
 - Medications
 - Precautions

Level 1 Data Flow Diagram – AI Healthcare System



4.2.3 System Architecture

The MediXAI system follows a **three-tier architecture**:

1. Presentation Layer (Frontend)

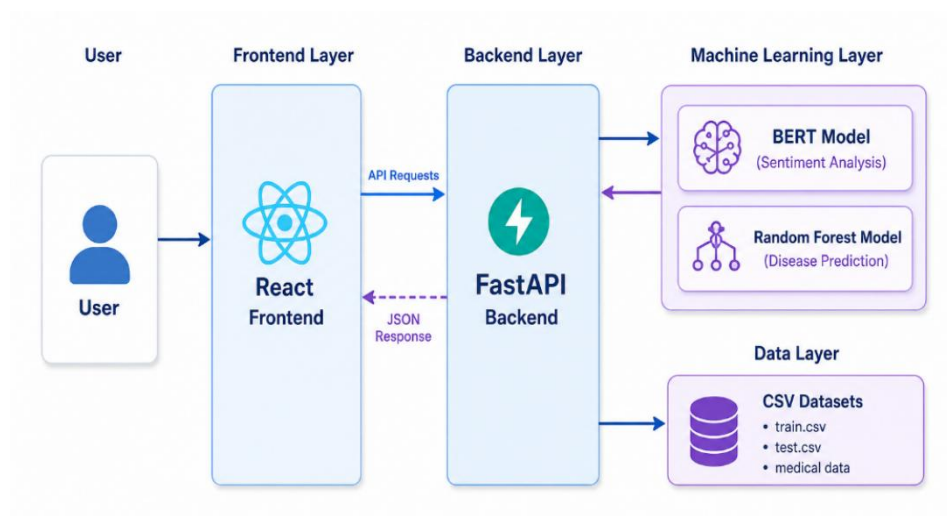
- Built using React.js
- Handles user interaction and UI rendering
- Sends API requests to backend

2. Application Layer (Backend)

- Built using FastAPI
- Handles request processing
- Manages business logic
- Interfaces with ML models

3. Data & Model Layer

- Contains:
 - BERT model (sentiment analysis)
 - Random Forest model (disease prediction)
 - CSV datasets (reviews and medical data)



4.2.4 Class Diagram (Conceptual)

Key components:

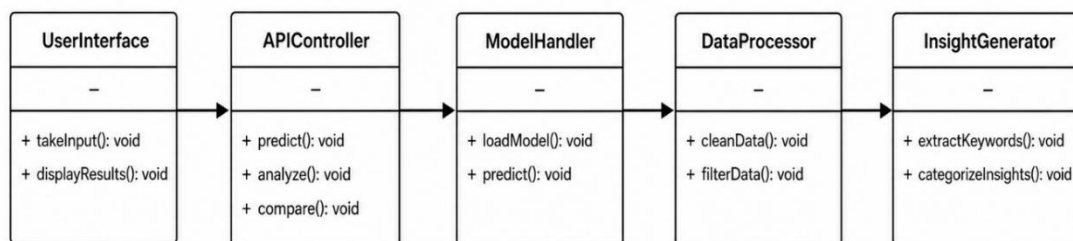
- **User Interface Class**
 - Handles input and output display
- **API Controller**
 - Manages endpoints (predict, analyze, compare)

- **Model Handler**
 - Loads and executes ML models
- **Data Processor**
 - Filters and processes dataset
- **Insight Generator**
 - Extracts benefits and limitations

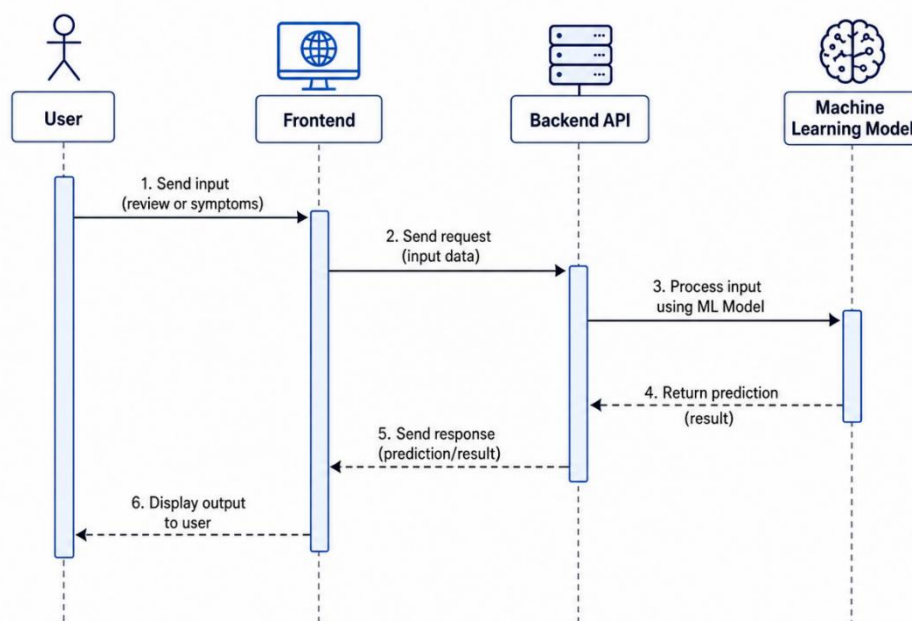
Relationships:

- UI → API Controller
- API Controller → Model Handler
- Model Handler → Data Processor
- Data Processor → Insight Generator

UML Class Diagram – AI-Based Web Application



UML Sequence Diagram – Web-Based AI System



4.2.5 Architecture Summary

The system is modular, scalable, and loosely coupled:

- Frontend and backend are independent
- Models are isolated for easy updates
- Data processing is separated from UI logic

This design ensures:

- Maintainability
- Scalability
- Flexibility for future enhancements

5. Appendices

5.1 Appendix A: Glossary

This section provides definitions of key terms used throughout the MediXAI system documentation to ensure clarity and consistency.

- **Artificial Intelligence (AI):** The simulation of human intelligence processes by machines, especially computer systems.
- **Machine Learning (ML):** A subset of AI that enables systems to learn from data and improve performance without explicit programming.
- **Natural Language Processing (NLP):** A field of AI that focuses on interaction between computers and human language.
- **BERT (Bidirectional Encoder Representations from Transformers):** A deep learning model used for understanding textual data, particularly for sentiment analysis.
- **Random Forest:** A supervised machine learning algorithm used for classification and prediction tasks, used here for disease prediction.
- **Explainable AI (XAI):** Techniques that make machine learning models transparent and interpretable.
- **SHAP (SHapley Additive exPlanations):** A method used to explain individual predictions of machine learning models.
- **API (Application Programming Interface):** A set of rules that allows different software components to communicate.
- **Frontend:** The user-facing part of the application built using React.js.
- **Backend:** The server-side logic implemented using FastAPI.
- **Dataset:** Structured data used for training and evaluating machine learning models.

- **Sentiment Analysis:** The process of determining the emotional tone (positive, neutral, negative) of text.
- **Symptom Extraction:** The process of identifying relevant symptoms from user input text.
- **Prediction Model:** A trained machine learning model used to generate outputs based on input data.

5.2 Appendix B: Analysis Model

The MediXAI system is built on a hybrid analytical model combining Natural Language Processing and Machine Learning techniques.

1. Sentiment Analysis Model (BERT)

- The system uses a pre-trained BERT (bert-base-uncased) model fine-tuned for classification.
- Input: Drug review text
- Output:
 - Sentiment label (Positive, Neutral, Negative)
 - Confidence score
- Process:
 - Tokenization of text
 - Encoding using transformer architecture
 - Classification using softmax layer

This model enables deep contextual understanding of user reviews.

2. Disease Prediction Model (Random Forest)

- A Random Forest classifier is used for predicting diseases based on symptoms.
- Input: Binary feature vector representing symptoms
- Output: Predicted disease label
- Process:
 - Symptom extraction from user input
 - Conversion into structured vector
 - Classification using ensemble decision trees

This model delivers robust, efficient predictive performance.

3. Insight Extraction Mechanism

- The system analyzes drug review datasets to extract:
 - Frequently occurring keywords
 - Benefits (positive indicators)
 - Limitations (side effects)
- Techniques used:
 - Text preprocessing (cleaning, tokenization)
 - Frequency analysis using Counter
 - Keyword grouping based on predefined categories

4. System Workflow Model

The overall workflow of the system is:

1. User input (review or symptoms)
2. Input processing and cleaning
3. Model inference (BERT or Random Forest)
4. Dataset lookup and insight generation
5. Response formatting
6. Display in frontend interface

This hybrid analytical model ensures both predictive accuracy and interpretability.

5.3 Appendix C: Issues List

This section outlines known limitations and challenges encountered during the development of the MediXAI system.

1. Model Complexity

- BERT-based models require high computational resources.
- SHAP explanations increase processing time significantly.

2. Deployment Constraints

- Free hosting platforms have limitations in handling heavy ML dependencies (e.g., SciPy, SHAP).
- Requires optimization for lightweight deployment.

3. Dataset Limitations

- Public datasets may not fully represent real-world diversity.
- Limited coverage of certain medical conditions and drugs.

4. Accuracy Limitations

- Predictions depend on quality and clarity of user input.
- Symptom-based predictions may not always be clinically accurate.

5. Input Variability

- Natural language input may include ambiguity or irrelevant information.
- Requires improved NLP preprocessing for robustness.

6. Explainability Trade-offs

- Removing SHAP for deployment reduces explainability depth.
- Trade-off between performance and interpretability.

7. No Real-Time Medical Validation

- System does not connect to live medical databases.
- Recommendations are based on static datasets.

8. Future Improvements

- Integration with real-time medical APIs
- Improved NLP for symptom extraction
- Deployment optimization for full model support
- Enhanced UI/UX for better accessibility